This query is a mashup of parameters, but here is the gist. We are searching for *Godfather* movies (`title`) starring Marlon Brando `OR` Al Pacino (`actors`) with a `rating` between 9 `AND` 9.5. We also add pagination (`from`, `size`), sort by rating, and ask Elasticsearch to `explain`. We retrieve two movies (both *Godfather*s), although Brando is not in *The Godfather Part II* (we specify `Brando OR Pacino` in the `actors` field).

As you may have guessed, the URL request method of writing queries is crude and error-prone. Ideally, we should write queries using Query DSL. Fortunately, we can wrap a URL request in Query DSL to get the best of both worlds.

### 8.6.4 Supporting URI requests with Query DSL

Query DSL has a `query_string` method that allows us to wrap a URI request call (we look at Query DSL in the next section). We can send the URI query parameters to search for movies with multiple title keywords in the request body as a `query_string`.

**Listing 8.11 Using a URI method wrapped in Query DSL**

```
GET movies/_search
{
  "query": {
    "query_string": {
      "default_field": "title",
      "query": "Knight Redemption Lord Pulp",
      "default_operator": "OR"
    }
  }
}
```

The `query_string` is equivalent to the `q` parameter we used earlier in the URI search method. While it is much better than the URI request approach, the `query_string` method is strict with syntax and has some unforgiving characteristics. Unless there's a strong reason not to, use Query DSL queries rather than `query_string`. We can use `query_string` for quick testing, but relying on it for complicated and in-depth queries may be asking for trouble.

It's time to venture into the mighty kingdom of Query DSL. Because Query DSL is a Swiss Army knife for searching, it warrants a dedicated section.

## 8.7 Query DSL

The Elasticsearch team developed the search-specific, all-purpose language and syntax called Query DSL. It's a sophisticated, powerful, expressive language that can create a multitude of queries from basic to complex, in addition to nested and more complicated queries. It can also be extended for analytical queries. It is a JSON-based query language that can be constructed with queries for search and analytics. The syntax and format are as follows:

```
GET books/_search
{
```

**Invokes the _search endpoint on the books index**